

DISCIPLINA • AULA 01

Integração e Entrega Contínua

Fundamentos de CI/CD: automação, qualidade e fluxo contínuo do código até a produção

Aula introdutória

Roteiro da aula

VISÃO GERAL

01

O problema

Integração tardia, conflitos e o "inferno da integração".

02

Conceitos

CI, Entrega Contínua e Implantação Contínua.

03

O pipeline

Estágios, gatilhos e feedback rápido.

04

Qualidade

Testes automatizados, análise estática e segurança.

05

Implantação

Blue-green, canário e rollback.

06

Medição

Métricas DORA, estudo de caso e síntese.

O problema: integração tardia

MOTIVAÇÃO

- Branches longos acumulam divergência por semanas.
- A integração vira um evento raro, manual e arriscado.
- Bugs aparecem tarde, quando corrigir custa mais caro.
- Testes manuais não acompanham o ritmo das mudanças.
- A entrega ao usuário fica presa a janelas esporádicas.

Custo de correção de um defeito

Codificação



Build/CI



Testes



Produção



Quanto mais tarde o defeito é detectado, maior o custo de correção.

O que é Integração Contínua (CI)

CONCEITO

Prática em que cada integração do trabalho ao repositório principal dispara automaticamente build e testes, expondo erros em minutos — não em semanas.

Frequência

Cada pessoa integra ao menos uma vez por dia.

Automação

Build e testes executam sem intervenção manual.

Feedback

O resultado chega em minutos a quem alterou o código.

Compromisso

Build quebrado é a prioridade máxima do time.

Práticas essenciais de CI

FUNDAMENTOS

Repositório único

Todo o código e configuração versionados em um só lugar.

Build automatizado

Um comando reproduz o build do zero em qualquer máquina.

Build auto-testável

A suíte de testes roda junto ao build e falha rápido.

Commits frequentes

Pequenos incrementos no tronco principal, todos os dias.

Ambiente-espelho

Testar em ambiente semelhante ao de produção.

Visibilidade

Estado do build visível para todo o time, o tempo todo.

Entrega Contínua x Implantação Contínua

DISTINÇÃO

Integração Contínua

Integrar, compilar e testar cada mudança automaticamente.

Resultado: código sempre integrado e verificado.

Entrega Contínua

Manter o software sempre pronto para produção; o deploy é decisão de negócio.

Resultado: release a um clique.

Implantação Contínua

Toda mudança aprovada no pipeline vai automaticamente à produção.

Resultado: deploy sem intervenção humana.

Anatomia de um pipeline

ARQUITETURA



Feedback rápido: qualquer falha interrompe o pipeline e notifica o time imediatamente.

Cada estágio recebe o artefato do anterior — o mesmo binário percorre todo o caminho até a produção.

Controle de versão e branching

BASE DO PIPELINE

- O repositório é a única fonte da verdade do projeto.
- Trunk-based: branches curtos, integrados em até um dia.
- GitFlow favorece releases planejados, mas atrasa a integração.
- Feature flags permitem entregar código incompleto com segurança.
- Pull requests pequenos aceleram revisão e reduzem conflitos.

Trunk-based

Branches de horas; integração diária no tronco.

GitFlow

Branches longos; integração tardia e mais conflitos.

Build automatizado e artefatos

AUTOMAÇÃO

Reprodutibilidade

O mesmo código gera sempre o mesmo artefato, em qualquer ambiente.

Dependências

Versões fixadas e resolvidas por gerenciador (Maven, npm, pip).

Build once

O artefato é construído uma única vez e promovido entre ambientes.

Rastreabilidade

Cada artefato é versionado e ligado ao commit que o originou.

Velocidade

Cache e paralelismo mantêm o build abaixo de dez minutos.

Containers

Imagens padronizam o ambiente de execução do build ao deploy.

Pirâmide de testes automatizados

QUALIDADE

Testes de interface (E2E)

Poucos, lentos e frágeis; validam fluxos críticos ponta a ponta.

Testes de integração/serviço

Verificam a colaboração entre componentes, APIs e banco de dados.

Testes de unidade

Muitos, rápidos e baratos; base de confiança do pipeline.

Princípios

- Falhar rápido e barato.
- Testes determinísticos, sem intermitência.
- Cobertura como indicador, não como meta.
- Rodar a cada commit.

Qualidade e segurança no pipeline

GUARDRAILS

Análise estática

Linters e SonarQube apontam falhas e dívida técnica antes do merge.

Quality gates

Limiars objetivos bloqueiam a promoção do artefato.

SCA de dependências

Varredura de vulnerabilidades em bibliotecas de terceiros.

Gestão de segredos

Credenciais em cofres, nunca no repositório.

DevSecOps

Segurança deslocada para o início (shift left) do ciclo.

Auditoria

Logs e evidências de cada execução do pipeline.

Estratégias de implantação

PRODUÇÃO

Recreate

Desliga a versão antiga e sobe a nova. Simples, mas com indisponibilidade.

Rolling update

Substitui instâncias gradualmente, mantendo o serviço no ar.

Blue-green

Dois ambientes idênticos; o tráfego alterna e o rollback é imediato.

Canário

Nova versão para uma fração de usuários, com expansão monitorada.

Rollback rápido e observabilidade são pré-requisitos para implantar com frequência e segurança.

Ecosistema de ferramentas

PANORAMA

Versionamento

Git, GitHub, GitLab, Bitbucket

Servidores de CI

Jenkins, GitHub Actions, GitLab CI, Azure Pipelines

Build e dependências

Maven, Gradle, npm, Make

Testes

JUnit, pytest, Jest, Selenium, Cypress

Qualidade e segurança

SonarQube, ESLint, Trivy, Dependabot

Deploy e infra

Docker, Kubernetes, Terraform, Ansible

Estudo de caso: API de pagamentos

PIPELINE REAL

Contexto

- Equipe de 8 pessoas, API em Node.js sobre Kubernetes.
- Repositório no GitHub, pipeline em GitHub Actions.
- Trunk-based com pull requests de até um dia.
- Deploy em homologação a cada merge no tronco.
- Produção por estratégia canário, com aprovação.

Antes do pipeline

Release	1 por mês, em fim de semana
Lead time	3 semanas do commit à produção
Testes	roteiro manual de 6 horas
Rollback	restauração manual do servidor

O pipeline na prática

ESTUDO DE CASO

```
workflow · ci.yml

on: [push, pull_request]
jobs:
  build:      # 1min  instala e compila
  unit:       # 2min  jest --coverage
  lint+sast:  # 1min  eslint, sonar, trivy
  integration:# 3min  docker compose + testes
  package:    # 1min  imagem versionada
  deploy-hml: # auto  após merge no main
  deploy-prd: # canário 5% -> 50% -> 100%
               needs: aprovação + métricas estáveis
```

Depois de 6 meses

Deploys

de 1/mês para 12/semana

Lead time

de 3 semanas para 2 dias

Feedback de CI

8 minutos por execução

Falha em mudanças

de 30% para 8%

Restauração

de 6 horas para 15 minutos

Números ilustrativos, coerentes com relatos típicos de adoção de CI/CD.

Como medir: métricas DORA

MEDICÃO

01

Frequência de implantação

Com que frequência a equipe entrega em produção.

02

Lead time para mudanças

Tempo do commit até a mudança rodar em produção.

03

Tempo de restauração

Quanto tempo leva para recuperar o serviço após falha.

04

Taxa de falha em mudanças

Percentual de implantações que causam incidentes.

As duas primeiras medem velocidade; as duas últimas, estabilidade. Times de alto desempenho melhoram ambas ao mesmo tempo.

Boas práticas e antipadrões

SÍNTESE

Faça

- Integrar ao tronco todos os dias.
- Manter o build verde como prioridade.
- Automatizar tudo o que se repete.
- Versionar também a infraestrutura.
- Medir e melhorar o pipeline continuamente.

Evite

- Branches que vivem semanas.
- Conviver com testes intermitentes.
- Pipelines lentos que ninguém espera.
- Etapas manuais entre build e deploy.
- Tratar CI/CD como problema só das ferramentas.

Quiz de fixação

VAMOS REVISAR

01 Qual a diferença entre Entrega Contínua e Implantação Contínua?

a) Nenhuma b) A aprovação humana antes da produção c) A linguagem usada

02 O que caracteriza a prática de Integração Contínua?

a) Ter um servidor de build b) Integrar ao tronco com frequência e testar automaticamente c) Fazer deploy diário

03 Na pirâmide de testes, quais devem ser mais numerosos?

a) Testes de interface b) Testes de integração c) Testes de unidade

04 Qual estratégia expõe a nova versão a uma fração dos usuários?

a) Recreate b) Canário c) Rolling update

05 Quais métricas DORA medem estabilidade?

a) Frequência e lead time b) Restauração e taxa de falha c) Cobertura e nº de commits

Responda individualmente em 5 minutos; depois discutimos as respostas em dupla.

Gabarito comentado

RESPOSTAS

01 – b

Na Entrega Contínua o artefato fica pronto e alguém decide liberar; na Implantação Contínua o pipeline libera sozinho.

02 – b

CI é a prática de integrar com frequência ao tronco com build e testes automáticos. A ferramenta é meio, não definição.

03 – c

Testes de unidade formam a base: rápidos, baratos e numerosos. Inverter a pirâmide gera pipelines lentos e instáveis.

04 – b

O canário libera a versão para uma fração dos usuários e expande conforme as métricas se mantêm saudáveis.

05 – b

Tempo de restauração e taxa de falha em mudanças medem estabilidade; frequência e lead time medem velocidade.

Dúvidas? Na próxima aula colocamos a mão na massa: criar um pipeline do zero.